

人工智能赋能的数字集成电路设计作业一

陈天乐

2025 年 11 月 18 日

目录

1	One Shot Prompt	2
2	Async FIFO Specification	9
3	RTL and TestBench	11
4	Simulation Result	19

1 One Shot Prompt

我在原本的 prompt 基础上添加了几个 Tips，并且加入了 Simulation Result, Last RTL, Last TestBench，分别是上一次代码的仿真结果、生成的 rtl 代码和 testbench 代码。

```

1 Below is a template for the problem input and response. You should reference this response
  format.
2
3 ---
4
5 Here is an examplaray prompt input:
6 Please help me generate a FIFO, including RTL code and TestBench.
7
8 ## Problem Description
9
10 ModelName: FIFO
11
12 InputNode:
13
14 - clk: Clock signal (rising edge effective).
15 - rst_n: Active-low reset signal.
16 - data_in: 32-bit input data.
17 - wr_en: Write enable signal, high effective.
18 - rd_en: Read enable signal, high effective.
19
20 OutputNode:
21
22 - data_out: 32-bit output data.
23 - full: FIFO full signal, high effective.
24 - empty: FIFO empty signal, high effective.
25
26 Specification:
27
28 - FIFO Size: 16 entries, each entry is 32 bits wide.
29 - Behavior:
30   - When wr_en is high and full is low, the input data data_in is written into the FIFO.
31   - When rd_en is high and empty is low, the oldest data in the FIFO is read out as data_out.
32   - If wr_en is high and full is high, the write operation is ignored.
33   - If rd_en is high and empty is high, the data_out remains unchanged.
34 - TestBench should contain the following test conditions:
35   - Basic Write and Read Operations: Write a few data entries and read them back in sequence.
36   - Concurrent Write and Read: Write and read data simultaneously to test FIFO behavior.
37   - FIFO Full Condition: Write data continuously until the FIFO is full, then attempt to write
    more data.
38   - FIFO Empty Condition: Read data continuously until the FIFO is empty, then attempt to read
    more data.
39   - Mixed Write and Read Patterns: Write and read data in various patterns to test FIFO
    robustness.
40   - Edge Cases: Write maximum number of entries and then read them back; read all entries and
    then write again.
41   - Reset Behavior: Test FIFO behavior after a reset signal is asserted.
42
43
44 Here is an examplaray prompt output:
45
46 ### RTL
47
48 ```verilog
49 module FIFO (
50     input wire clk,
51     input wire rst_n,
52     input wire [31:0] data_in,

```

```

53     input wire wr_en,
54     input wire rd_en,
55     output reg [31:0] data_out,
56     output wire full,
57     output wire empty
58 );
59
60     // FIFO parameters
61     parameter DEPTH = 16;
62     parameter WIDTH = 32;
63
64     // Internal memory
65     reg [WIDTH-1:0] memory [0:DEPTH-1];
66
67     // Pointers
68     reg [3:0] wr_ptr;
69     reg [3:0] rd_ptr;
70     reg [4:0] count; // Need 5 bits to count 0-16
71
72     // Full and empty flags
73     assign full = (count == DEPTH);
74     assign empty = (count == 0);
75
76     // FIFO control logic
77     always @(posedge clk or negedge rst_n) begin
78         if (!rst_n) begin
79             // Reset all pointers and memory
80             wr_ptr <= 0;
81             rd_ptr <= 0;
82             count <= 0;
83             data_out <= 0;
84
85             // Optional: Clear memory on reset
86             for (integer i = 0; i < DEPTH; i = i + 1) begin
87                 memory[i] <= 0;
88             end
89         end else begin
90             // Write operation
91             if (wr_en && !full) begin
92                 memory[wr_ptr] <= data_in;
93                 wr_ptr <= wr_ptr + 1;
94                 if (wr_ptr == DEPTH-1) wr_ptr <= 0;
95             end
96
97             // Read operation
98             if (rd_en && !empty) begin
99                 data_out <= memory[rd_ptr];
100                 rd_ptr <= rd_ptr + 1;
101                 if (rd_ptr == DEPTH-1) rd_ptr <= 0;
102             end
103
104             // Update count
105             case ({wr_en && !full, rd_en && !empty})
106                 2'b00: count <= count; // No operation
107                 2'b01: count <= count - 1; // Read only
108                 2'b10: count <= count + 1; // Write only
109                 2'b11: count <= count; // Simultaneous read and write
110             endcase
111         end
112     end
113
114 endmodule

```

```

115 ```
116
117 ### TestBench
118
119 ```systemverilog
120 module FIFO_tb;
121     // Parameters
122     parameter CLK_PERIOD = 10;
123     parameter DEPTH = 16;
124
125     // Signals
126     reg clk;
127     reg rst_n;
128     reg [31:0] data_in;
129     reg wr_en;
130     reg rd_en;
131     wire [31:0] data_out;
132     wire full;
133     wire empty;
134
135     // Test variables
136     reg [31:0] expected_data;
137     integer error_count;
138     integer test_case;
139
140     // Instantiate DUT
141     FIFO dut (
142         .clk(clk),
143         .rst_n(rst_n),
144         .data_in(data_in),
145         .wr_en(wr_en),
146         .rd_en(rd_en),
147         .data_out(data_out),
148         .full(full),
149         .empty(empty)
150     );
151
152     // Clock generation
153     always #(CLK_PERIOD/2) clk = ~clk;
154
155     // Waveform dumping
156     initial begin
157         $dumpfile("Out/wave.vcd");
158         $dumpvars(0, FIFO_tb);
159     end
160
161     // Test tasks
162     task write_data(input [31:0] data);
163         begin
164             data_in = data;
165             wr_en = 1;
166             @(posedge clk);
167             wr_en = 0;
168         end
169     endtask
170
171     task read_data(output [31:0] data);
172         begin
173             rd_en = 1;
174             @(posedge clk);
175             rd_en = 0;
176             data = data_out;

```

```

177     end
178 endtask
179
180 task reset_fifo;
181     begin
182         rst_n = 0;
183         wr_en = 0;
184         rd_en = 0;
185         data_in = 0;
186         @(posedge clk);
187         @(posedge clk);
188         rst_n = 1;
189         @(posedge clk);
190     end
191 endtask
192
193 task check_result(input [31:0] actual, input [31:0] expected, input string test_name);
194     begin
195         if (actual != expected) begin
196             $display("ERROR: %s - Expected: %h, Got: %h", test_name, expected, actual);
197             error_count = error_count + 1;
198         end
199     end
200 endtask
201
202 // Main test sequence
203 initial begin
204     // Initialize
205     clk = 0;
206     error_count = 0;
207     test_case = 0;
208
209     // Test 1: Reset behavior
210     test_case = 1;
211     $display("Test %0d: Reset Behavior", test_case);
212     reset_fifo();
213     check_result(data_out, 32'h0, "Reset data_out");
214     check_result(empty, 1'b1, "Reset empty flag");
215     check_result(full, 1'b0, "Reset full flag");
216
217     // Test 2: Basic write and read operations
218     test_case = 2;
219     $display("Test %0d: Basic Write and Read Operations", test_case);
220     reset_fifo();
221
222     // Write 4 data items
223     write_data(32'h12345678);
224     write_data(32'hABCDEF01);
225     write_data(32'hDEADBEEF);
226     write_data(32'hCAFEBABE);
227
228     // Read them back
229     read_data(expected_data);
230     check_result(expected_data, 32'h12345678, "First read");
231     read_data(expected_data);
232     check_result(expected_data, 32'hABCDEF01, "Second read");
233     read_data(expected_data);
234     check_result(expected_data, 32'hDEADBEEF, "Third read");
235     read_data(expected_data);
236     check_result(expected_data, 32'hCAFEBABE, "Fourth read");
237
238     // Test 3: Concurrent write and read

```

```

239     test_case = 3;
240     $display("Test %0d: Concurrent Write and Read", test_case);
241     reset_fifo();
242
243     fork
244     begin
245         for (integer i = 0; i < 8; i = i + 1) begin
246             write_data(32'h1000 + i);
247         end
248     end
249     begin
250         for (integer i = 0; i < 8; i = i + 1) begin
251             read_data(expected_data);
252             check_result(expected_data, 32'h1000 + i, $sformatf("Concurrent read %0d",
253 i));
254         end
255     end
256     join
257
258     // Test 4: FIFO full condition
259     test_case = 4;
260     $display("Test %0d: FIFO Full Condition", test_case);
261     reset_fifo();
262
263     // Fill the FIFO
264     for (integer i = 0; i < DEPTH; i = i + 1) begin
265         write_data(32'h2000 + i);
266         if (i == DEPTH-1) begin
267             check_result(full, 1'b1, "FIFO should be full");
268         end
269     end
270
271     // Try to write when full (should be ignored)
272     data_in = 32'hBAD0BAD0;
273     wr_en = 1;
274     @(posedge clk);
275     wr_en = 0;
276     @(posedge clk);
277     check_result(full, 1'b1, "FIFO should remain full");
278
279     // Test 5: FIFO empty condition
280     test_case = 5;
281     $display("Test %0d: FIFO Empty Condition", test_case);
282
283     // Empty the FIFO
284     for (integer i = 0; i < DEPTH; i = i + 1) begin
285         read_data(expected_data);
286         check_result(expected_data, 32'h2000 + i, $sformatf("Emptying read %0d", i));
287         if (i == DEPTH-1) begin
288             check_result(empty, 1'b1, "FIFO should be empty");
289         end
290     end
291
292     // Try to read when empty
293     rd_en = 1;
294     @(posedge clk);
295     rd_en = 0;
296     @(posedge clk);
297     check_result(empty, 1'b1, "FIFO should remain empty");
298
299     // Test 6: Mixed write and read patterns
300     test_case = 6;

```

```

300     $display("Test %0d: Mixed Write and Read Patterns", test_case);
301     reset_fifo();
302
303     // Pattern 1: Write 3, read 2, write 2, read 3
304     write_data(32'h3001); write_data(32'h3002); write_data(32'h3003);
305     read_data(expected_data); check_result(expected_data, 32'h3001, "Mixed pattern 1a");
306     read_data(expected_data); check_result(expected_data, 32'h3002, "Mixed pattern 1b");
307     write_data(32'h3004); write_data(32'h3005);
308     read_data(expected_data); check_result(expected_data, 32'h3003, "Mixed pattern 1c");
309     read_data(expected_data); check_result(expected_data, 32'h3004, "Mixed pattern 1d");
310     read_data(expected_data); check_result(expected_data, 32'h3005, "Mixed pattern 1e");
311
312     // Test 7: Edge cases
313     test_case = 7;
314     $display("Test %0d: Edge Cases", test_case);
315     reset_fifo();
316
317     // Write maximum entries and read them back
318     for (integer i = 0; i < DEPTH; i = i + 1) begin
319         write_data(32'h4000 + i);
320     end
321     check_result(full, 1'b1, "Edge case - full");
322
323     for (integer i = 0; i < DEPTH; i = i + 1) begin
324         read_data(expected_data);
325         check_result(expected_data, 32'h4000 + i, $sformatf("Edge case read %0d", i));
326     end
327     check_result(empty, 1'b1, "Edge case - empty");
328
329     // Write again after reading all
330     write_data(32'h5001);
331     read_data(expected_data);
332     check_result(expected_data, 32'h5001, "Write after complete read");
333
334     // Final result
335     if (error_count == 0) begin
336         $display("TestSucceeded: All tests passed!");
337     end else begin
338         $display("TestFailed: %0d errors found", error_count);
339     end
340
341     $finish;
342 end
343
344 endmodule
345 ```
346
347 ---
348
349 Here is the real input prompt
350
351 Please help me generate [ModelName], including RTL code and TestBench.
352
353 ## Problem Description
354
355 ModelName: [ModelName]
356
357 InputNode: [InputNode]
358
359 OutputNode: [OutputNode]
360
361 Specification: [Specification]

```

```
362
363 TestBench: [TestSpec]
364
365 Last_RTL: [LastRTL]
366
367 Last_TestBench: [LastTestBench]
368
369 SimulationResult: [SimulationResult]
370
371
372 ### Tips
373
374 Here is some tips that you need to follow:
375
376 1. Use Verilog for RTL and SystemVerilog for TestBench.
377 2. We use the -g2005sv specification supported by iverilog for compilation. Please make sure
    that the verilog used can be supported. Please ensure that the system functions are
    supported by the -g2005sv. Some functions like $sformatf() will arise error.
378 3. Unpacked structs not supported in iverilog, do not use typedef struct.
379 4. When using the $bitstoreal function, please make sure that the parameter is 64bit.
380 5. Save the simulated waveform diagram with $dumpfile and $dumpvars instructions. Save the vcd
    file as Out/wave.vcd
381 6. In Testbench, please verify whether the function is correct. If you pass all the tests,
    please output TestSuccesed, otherwise you will output TestFailed.
382 7. In Verilog, for loops cannot be used to initialize arrays within always blocks for
    sequential logic that are triggered by a clock.
383 8. In Verilog, earlier versions (such as those prior to Verilog-2001) do not support directly
    declaring loop variables in the initialization section of a for loop. This is an vital
    part, so you should pay attention to this point. Otherwise the simulator will call error:
    name is not a valid net. You can define it previously.
384 9. Last_RTL, Last_TestBench, and SimulationResult refer to the previously generated RTL code,
    TestBench code, and the error results from the simulator. Please adjust the newly
    generated code by incorporating these, and make every effort to eliminate the errors.
385 10. When executing the Testbench, first print out the line: "### SimulationResult\n ''
    markdown\n" in testbench.
```

2 Async FIFO Specification

```

"model_name": "async_fifo",
"input_node": "- wr_clk: Write clock domain clock signal (rising edge effective).\n rd_clk: Read clock domain clock signal (rising edge effective).\n rst_n: Active-low asynchronous reset signal.\n wr_en: Write enable signal, high effective.\n rd_en: Read enable signal, high effective.\n data_in: 16-bit input data to be written to FIFO.",
"output_node": "- data_out: 16-bit output data read from FIFO.\n full: FIFO full indicator, high effective. Only asserted when the write-domain count (wr_count) equals the depth (16), not using '>' depth.\n empty: FIFO empty indicator, high effective. Only asserted when the read-domain count (rd_count) equals 0, with no early or delayed assertion.\n almost_full: FIFO almost full indicator (programmable threshold), high effective.\n almost_empty: FIFO almost empty indicator (programmable threshold), high effective.",
"specification": "- Support IEEE 754 Half-Precision Floating-Point Format (FP16).\n Support asynchronous clock domains (wr_clk and rd_clk) are independent, with arbitrary frequencies and phases).\n Use Gray code-encoded read and write pointers, synchronized across clock domains via two-stage synchronizers to avoid metastability.\n FIFO depth is 16 (can store 16 16-bit data words). Programmable almost-full threshold (default: 14) and almost-empty threshold (default: 2).\n Asynchronous reset behavior: When rst_n is low, all read/write pointers are cleared, full/empty/almost_full/almost_empty flags are reset, and the FIFO is initialized to empty.\n Data write: When wr_en is high and full is low, data_out is read from the FIFO on the rising edge of wr_clk. Never perform read operations when empty (causes pointer misalignment).\n Data read: When rd_en is high and empty is low, data_out is read from the FIFO on the rising edge of rd_clk, following first-in-first-out (FIFO) order.\n Full flag generation: Asserted in the write clock domain when the FIFO is completely full (write pointer catches up to the synchronized write pointer in Gray code).\n Empty flag generation: Asserted in the read clock domain when the FIFO is completely empty (read pointer catches up to the synchronized write pointer in Gray code).\n Almost-full flag: Asserted when the number of occupied entries reaches the almost-full threshold (programmable, default 14).\n Almost-empty flag: Asserted when the number of occupied entries drops to the almost-empty threshold (programmable, default 2).\n Traditional Verilog has limited support for loop-based assignments to reg arrays. The syntax of iterating through an array and assigning values via a for loop within an always block is not allowed in older standards. Therefore, if you need to initialize and clear the memory of this FIFO, you either place the initialization in an initial block (for simulation purposes) or use other methods to clear the memory.\n The synchronizer delay from the write pointer to the read pointer in the asynchronous FIFO causes the read domain to fail to timely detect that the FIFO is not empty, thus losing the first written data. Therefore, it is necessary to ensure that the start time of the read operation is later than the synchronizer's stable time, or to optimize the synchronization logic to reduce the impact of delay on the critical path.\n Read and write pointers use n+1 bits (e.g., 5 bits for a depth of 16, where the highest 1 bit serves as the carry bit and the lower 4 bits as the address). When calculating FIFO counts (fifo_count_wr_domain/fifo_count_rd_domain), it is mandatory to check whether the highest 1 bit of the pointers is consistent: 1. If the highest 1 bit is the same, the count equals the difference of the lower n bits; 2. If the highest 1 bit is different (pointer wrap-around), the count equals the difference of the lower n bits plus the depth. Directly calculating the count using wr_ptr_bin - rd_ptr_bin_sync or similar methods is prohibited

```

```

memory(wr_ptr_bin[3:0]) <= data_in, wr_ptr_bin <= wr_ptr_bin + 1,
end, end, end, end, // Read control logic, always @ (posedge
rd_clk or negedge rst_n) begin
if (rst_n == 0) begin
data_out <= 0;
end else begin
if (rd_en && !empty) begin
data_out <= memory[rd_ptr_bin[3:0]];
end, end, end, end, // Initialize memory (for simulation only)
integer init_i;
initial begin
for (init_i = 0; init_i < DEPTH; init_i = init_i + 1)
begin
memory[init_i] = 0;
end, end, end, end, // Parameters
parameter
WR_CLK_PERIOD = 10, // 100MHz
parameter RD_CLK_PERIOD = 20, // 50MHz
parameter DEPTH = 16, // Signals
reg wr_clk, rd_clk, reg
rst_n, reg wr_en, reg rd_en, reg [15:0] data_in, reg [15:0] data_out,
wire full, wire empty, wire almost_full, wire almost_empty, //
Test variables
reg [15:0] expected_data, integer error_count, integer
test_case, integer i;
// Instantiate DUT
async_fifo dut
(
wr_clk(wr_clk),
rd_clk(rd_clk),
rst_n(rst_n),
wr_en(wr_en),
rd_en(rd_en),
data_in(data_in),
data_out(data_out),
full(full),
empty(empty),
almost_full(almost_full),
almost_empty(almost_empty)
);
// Clock generation
always # (WR_CLK_PERIOD/2) wr_clk = ~wr_clk;
always # (RD_CLK_PERIOD/2) rd_clk = ~rd_clk;
// Waveform dumping
initial begin
$dumpvars(0, async_fifo_tb);
end, // Test tasks
task write_data(input
[15:0] data);
begin
data_in = data;
wr_en = 1;
@ (negedge wr_clk); // Ensure wr_en setup time
@ (posedge wr_clk);
wr_en = 0;
end, end, // task read_data(output [15:0] data);
begin
rd_en = 1;
@ (posedge rd_clk);
rd_en = 0;
data = data_out;
end, end, // task
reset_fifo;
begin
rst_n = 0;
wr_en = 0;
rd_en = 0;
data_in = 0;
@ (posedge wr_clk);
@ (posedge wr_clk);
@ (posedge rd_clk);
@ (posedge rd_clk);
rst_n = 1;
end, end, // task check_result(input actual, input expected, input string
test_name);
begin
if (actual != expected) begin
$display("ERROR: %s - Expected: %b, Got: %b", test_name, expected, actual);
error_count = error_count + 1;
end, end, // task check_data(input [15:0] actual, input [15:0] expected, input string test_name);
begin
if (actual != expected) begin
$display("ERROR: %s - Expected: %h, Got: %h", test_name, expected, actual);
error_count = error_count + 1;
end, end, // Main test
sequence()
initial begin
// Initialize
wr_clk = 0;
rd_clk = 0;
error_count = 0;
test_case = 0;
end, // Test 1:
SimulationResult();
$display("markdown");
// Test 1:
Reset behavior
test_case = 1;
$display("Test %0d: Reset Behavior",
test_case);
reset_fifo;
check_data_result(data_out, 16'h0, "Reset

```

```

// (failure to handle wrap-around will result in errors in full/empty flags).
"rtl_code": "module async_fifo (
input wire rst_n,
input wire wr_clk,
input wire rd_clk,
input wire rst_n,
input wire wr_en,
input wire rd_en,
input wire [15:0] data_in,
output reg [15:0] data_out,
output wire full,
output wire empty,
output wire almost_full,
output wire almost_empty, // Parameters
parameter DEPTH = 16,
parameter ALMOST_FULL_THRESHOLD = 14,
parameter ALMOST_EMPTY_THRESHOLD = 2, // Internal memory
reg [WIDTH-1:0] memory [0:DEPTH-1]; // Pointers (5 bits for depth 16)
reg [4:0] rd_ptr_bin, reg [4:0] rd_ptr_gray, reg [4:0] wr_ptr_gray_sync1, wr_ptr_gray_sync2, reg [4:0] rd_ptr_gray_sync1, rd_ptr_gray_sync2, // Binary synchronized pointers
reg [4:0] wr_ptr_bin_sync, reg [4:0] rd_ptr_bin_sync, // FIFO counts
reg [4:0] fifo_count_wr_domain, reg [4:0] fifo_count_rd_domain, // Convert binary to Gray code
assign wr_ptr_gray = wr_ptr_bin ^ (wr_ptr_bin >> 1);
assign rd_ptr_gray = rd_ptr_bin ^ (rd_ptr_bin >> 1); // Convert Gray code to binary
function [4:0] gray2bin;
input [4:0] gray;
begin
gray2bin[4] = gray[4];
gray2bin[3] = gray[4] ^ gray[3];
gray2bin[2] = gray[4] ^ gray[3] ^ gray[2];
gray2bin[1] = gray[4] ^ gray[3] ^ gray[2] ^ gray[1];
gray2bin[0] = gray[4] ^ gray[3] ^ gray[2] ^ gray[1] ^ gray[0]; // Write pointer synchronization (to read domain)
always @ (posedge rd_clk or negedge rst_n)
begin
if (rst_n) begin
wr_ptr_gray_sync1 <= 0;
wr_ptr_bin_sync <= 0;
end else begin
wr_ptr_gray_sync1 <= wr_ptr_gray;
wr_ptr_bin_sync <= gray2bin(wr_ptr_gray_sync1);
wr_ptr_bin_sync <= gray2bin(wr_ptr_gray_sync2);
always @ (posedge wr_clk or negedge rst_n) begin
if (rst_n) begin
rd_ptr_gray_sync1 <= 0;
rd_ptr_gray_sync2 <= 0;
rd_ptr_bin_sync <= 0;
end else begin
rd_ptr_gray_sync1 <= rd_ptr_gray;
rd_ptr_gray_sync2 <= rd_ptr_gray;
rd_ptr_bin_sync <= gray2bin(rd_ptr_gray_sync2);
end, // Calculate FIFO count in write domain
always @ (*) begin
if (wr_ptr_bin[4] == rd_ptr_bin_sync[4]) begin
fifo_count_wr_domain = wr_ptr_bin[3:0] - rd_ptr_bin_sync[3:0];
end else begin
fifo_count_wr_domain = DEPTH - rd_ptr_bin_sync[3:0] + wr_ptr_bin[3:0];
end, // Calculate FIFO count in read domain
always @ (*) begin
if (wr_ptr_bin_sync[4] == rd_ptr_bin[4]) begin
fifo_count_rd_domain = wr_ptr_bin_sync[3:0] - rd_ptr_bin[3:0];
end else begin
fifo_count_rd_domain = DEPTH - rd_ptr_bin[3:0] + wr_ptr_bin_sync[3:0];
end, // Generate flags
assign full = (fifo_count_wr_domain == DEPTH);
assign empty = (fifo_count_rd_domain == 0);
assign almost_full = (fifo_count_wr_domain == ALMOST_FULL_THRESHOLD);
assign almost_empty = (fifo_count_rd_domain == ALMOST_EMPTY_THRESHOLD); // Write control logic
always @ (posedge wr_clk or negedge rst_n) begin
integer i;
if (rst_n) begin
wr_ptr_bin <= 0;
end else begin
if (wr_en && !full) begin

```

```

data_out");
check_result(empty, 1'b1, "Reset empty flag");
check_result(full, 1'b0, "Reset full flag");
check_result(almost_empty, 1'b1, "Reset almost empty flag");
check_result(almost_full, 1'b0, "Reset almost full flag");
if (error_count == 0) begin
$display("Test %0d: Reset Behavior - PASSED", test_case);
end, // Test 2: Basic write and read operations
test_case = 2;
$display("Test %0d: Basic Write and Read Operations", test_case);
reset_fifo;
// Write 4 data items
write_data(16'h1234);
write_data(16'hABCD);
write_data(16'h5678);
write_data(16'hFE01);
// Add sufficient delay for synchronization (more cycles for async FIFO)
repeat(8) @ (posedge rd_clk);
// Read them back
read_data(expected_data);
check_data_result(expected_data, 16'h1234, "First read");
read_data(expected_data);
check_data_result(expected_data, 16'hABCD, "Second read");
read_data(expected_data);
check_data_result(expected_data, 16'h5678, "Third read");
read_data(expected_data);
check_data_result(expected_data, 16'hFE01, "Fourth read");
if (error_count == 0) begin
$display("Test %0d: Basic Write and Read Operations - PASSED", test_case);
end, // Test 3: Asynchronous clock test
test_case = 3;
$display("Test %0d: Asynchronous Clock Test", test_case);
reset_fifo;
for (i = 0; i < 8; i = i + 1) begin
write_data(16'h1000 + i);
end, // Add more delay to ensure first data is written and synchronized
repeat(10) @ (posedge rd_clk);
for (i = 0; i < 8; i = i + 1) begin
read_data(expected_data);
check_data_result(expected_data, 16'h1000 + i, "Async read");
end, // Test 4: FIFO full condition
if (error_count == 0) begin
$display("Test %0d: Asynchronous Clock Test - PASSED", test_case);
end, // Test 4: FIFO full condition
test_case = 4;
$display("Test %0d: FIFO Full Condition", test_case);
reset_fifo;
// Fill the FIFO first
for (i = 0; i < DEPTH; i = i + 1) begin
write_data(16'h2000 + i);
end, // FIFO should be full
check_result(full, 1'b1, "FIFO should be full");
check_result(almost_full, 1'b1, "FIFO should be almost full");
// Try to write when full (should be ignored)
data_in = 16'hBADC;
wr_en = 1;
@ (negedge wr_clk);
@ (posedge wr_clk);
wr_en = 0;
if (error_count == 0) begin
check_result(full, 1'b1, "FIFO should remain full");
end, // Test 5: FIFO empty condition
test_case = 5;
$display("Test %0d: FIFO Empty Condition", test_case);
reset_fifo;
// Fill the FIFO first
for (i = 0; i < DEPTH; i = i + 1) begin
write_data(16'h2000 + i);
end, // Add delay for synchronization
repeat(8) @ (posedge rd_clk);
// Empty the FIFO
for (i = 0; i < DEPTH; i = i + 1) begin
read_data(expected_data);
check_data_result(expected_data, 16'h2000 + i, "Emptying read");
if (i == DEPTH-1) begin

```

```

check_result(empty, 1'b1, "FIFO should be empty");
check_result(almost_empty, 1'b1, "FIFO should be almost empty");
endn
endn // Try to read when empty
rd_en = 1;
@posedge rd_clk
rd_en = 0;
check_result(empty, 1'b1, "FIFO should remain empty");
if (error_count == 0)
begin
$display("Test %0d: FIFO Empty Condition - PASSED", test_case);
endn
// Test 6: Almost-Full/Almost-empty threshold test
test_case = 6;
$display("Test %0d: Almost-Full/Almost-empty Threshold Test");
test_case = 6;
reset_fifo();
// Write until almost full threshold
for (i = 0; i < 13; i = i + 1) begin
write_data(16'h3000 + i);
endn
check_result(almost_full, 1'b0, "Should not be almost full yet");
write_data(16'h3000 + 13);
check_data_result(expected_data, 16'h3000 + i,
\Threshold read");
endn // After reading 12 from 14, we have 2 left -
should be almost empty
check_result(almost_empty, 1'b1, "Should be almost empty
at 2 entries");
check_result(empty, 1'b0, "Should not be empty yet");
if (error_count == 0) begin
$display("Test %0d:
Almost-Full/Almost-empty Threshold Test - PASSED", test_case);
endn
// Test 7: Concurrent read-write test
test_case = 7;
$display("Test %0d:
Concurrent Read-Write Test", test_case);
reset_fifo();
begin
for (i = 0; i < 10; i = i + 1) begin
write_data(16'h4000 + i);
endn
endn
// Add more delay to ensure first data is written and synchronized
repeat(12)
@posedge rd_clk
read_data(expected_data);
check_data_result(expected_data,
16'h4000 + i, "Concurrent read");
join
if (error_count == 0) begin
$display("Test %0d: Concurrent
Read-Write Test - PASSED", test_case);
endn
// Test 8: Edge
case test
test_case = 8;
$display("Test %0d: Edge Case Test");
test_case = 8;
reset_fifo();
// Write 15 entries (one before full)
for (i = 0; i < 15; i = i + 1) begin
write_data(16'h5000 + i);
endn
check_result(full, 1'b0, "Should not be full at 15 entries");
check_result(almost_full,
1'b1, "Should be almost full at 15 entries");
write_data(16'h5000 + 15);
check_result(full, 1'b1, "Should be full at 16 entries");
// Add delay for synchronization
repeat(8) @posedge rd_clk
// Read 14 entries - should have 2 left (almost empty)
for (i = 0; i < 14; i = i
+ 1) begin
read_data(expected_data);
check_data_result(expected_data, 16'h5000 + i, "Edge case read");
endn
check_result(almost_empty, 1'b1, "Should be almost empty at 2 entries");
endn

```

```

// Internal memory
reg [WIDTH-1:0] memory [0:DEPTH-1]; // Pointers (5 bits
for depth 16)
reg [4:0] wr_ptr_bin, rd_ptr_bin; reg [4:0] wr_ptr_gray, rd_ptr_gray;
// Synchronized pointers
reg [4:0] wr_ptr_gray_sync1, wr_ptr_gray_sync2;
rd_ptr_gray_sync1, rd_ptr_gray_sync2;
// Binary synchronized pointers
reg [4:0] wr_ptr_bin_sync1, wr_ptr_bin_sync2;
rd_ptr_bin_sync1, rd_ptr_bin_sync2;
// FIFO counts
reg [4:0] fifo_count_wr_domain;
reg [4:0] fifo_count_rd_domain;
// Convert binary to
Gray code
assign wr_ptr_gray = wr_ptr_bin ^ (wr_ptr_bin >> 1);
assign rd_ptr_gray = rd_ptr_bin ^ (rd_ptr_bin >> 1);
// Convert Gray code to binary
function [4:0]
gray2bin;
input [4:0] gray;
begin
gray2bin[0] = gray[0];
gray2bin[1] = gray[4] ^ gray[3];
gray2bin[2] = gray[4] ^ gray[3] ^ gray[2];
gray2bin[3] = gray[4] ^ gray[3] ^ gray[2] ^ gray[1];
gray2bin[4] = gray[4] ^ gray[3] ^ gray[2] ^ gray[1] ^ gray[0];
endn
endfunction
// Write pointer
synchronization (to read clock domain)
always @posedge wr_clk or negedge rst_n
begin
if (!rst_n) begin
wr_ptr_gray_sync1 <= 0;
wr_ptr_gray_sync2 <= 0;
wr_ptr_bin_sync1 <= wr_ptr_gray;
wr_ptr_bin_sync2 <= gray2bin(wr_ptr_gray_sync1);
endn
endn
// Read pointer
synchronization (to write clock domain)
always @posedge wr_clk or
negedge rst_n
begin
if (!rst_n) begin
rd_ptr_gray_sync1 <= 0;
rd_ptr_gray_sync2 <= 0;
rd_ptr_bin_sync1 <= rd_ptr_gray;
rd_ptr_bin_sync2 <= gray2bin(rd_ptr_gray_sync1);
endn
endn
// Calculate FIFO count in write domain
always @(*) begin
if (wr_ptr_bin[4] ==
rd_ptr_bin_sync1[4]) begin
fifo_count_wr_domain = wr_ptr_bin[3:0] -
rd_ptr_bin_sync1[3:0];
endn
endn
// Calculate
FIFO count in read domain
always @(*) begin
if (wr_ptr_bin_sync1[4] ==
rd_ptr_bin[4]) begin
fifo_count_rd_domain = wr_ptr_bin_sync1[3:0] -
rd_ptr_bin[3:0];
endn
endn
// Generate
flags
assign full = (fifo_count_wr_domain == DEPTH);
assign almost_full = (fifo_count_wr_domain >=
ALMOST_FULL_THRESHOLD);
assign almost_empty = (fifo_count_rd_domain <=
ALMOST_EMPTY_THRESHOLD);
// Write control logic
always @posedge
wr_clk or negedge rst_n
begin
if (wr_en && !full) begin
memory[wr_ptr_bin[3:0]] <= data_in;
wr_ptr_bin <= wr_ptr_bin + 1;
endn
endn
// Read control logic
always @posedge
rd_clk or negedge rst_n
begin
if (rd_en && !empty) begin
data_out <= memory[rd_ptr_bin[3:0]];
rd_ptr_bin <= rd_ptr_bin + 1;
endn
endn
// Initialize memory (for simulation only)
integer init_i;
initial begin
for (init_i = 0; init_i < DEPTH; init_i = init_i + 1)

```

```

// Read 1 more entry - should have 1 left (still almost empty)
read_data(expected_data);
check_result(almost_empty, 1'b1, "Should still be almost
empty at 1 entry");
// Read last entry - should be empty
read_data(expected_data);
check_result(empty, 1'b1, "Should be empty after
reading all");
check_result(almost_empty, 1'b1, "Should be almost empty at 0
entries");
if (error_count == 0) begin
$display("Test %0d: Edge Case Test - PASSED", test_case);
endn
// Final result
if (error_count == 0) begin
$display("Test %0d: Success");
endn
All tests passed
end else begin
$display("Test %0d: Failed");
endn
$finish;
endn
endmodule

```

"test_spec": "- Basic Read-Write Test: Write a sequence of data (e.g., 0x0000 to 0x000F) and read them back, verifying FIFO order and data integrity.\n- Asynchronous Clock Test: Configure wr_clk at 100MHz and rd_clk at 50MHz, perform continuous write and read operations, verify data correctness across clock domains.\n- Full Condition Test: Continuously write data until full is asserted, check that no more data can be written, and verify full flag accuracy.\n- Empty Condition Test: Continuously read data until empty is asserted, check that no more data can be read, and verify empty flag accuracy.\n- Almost-Full/Almost-Empty Threshold Test: Set custom thresholds (e.g., almost_full=10, almost_empty=3), verify that the flags are asserted correctly when the entry count reaches the thresholds.\n- Reset Test: Assert rst_n during operation, check that all pointers, flags, and data outputs are initialized correctly after reset.\n- Concurrent Read-Write Test: Perform write and read operations simultaneously (with proper clock domain handling), verify FIFO stability and data consistency.\n- Edge Case Test: Write 15 entries (one before full), write 16 entries (full), read 1 entry (almost empty at threshold), read all entries (empty), and verify corresponding flag behaviors.\n- Once a test case passes, please print it out.\n- Ensure wr_en is asserted after the wr_clk negedge (falling edge) and remains high until the next wr_clk posedge (rising edge). This guarantees wr_en has sufficient setup time (at least half the wr_clk period) before the rising edge, so the write logic reliably detects wr_en=1 on the posedge."

"RelatedID": null,
"SimulationResult": "Test 1: Reset Behavior\nTest 1: Reset Behavior - PASSED\nTest 2: Basic Write and Read Operations\nTest 2: Basic Write and Read Operations - PASSED\nTest 3: Asynchronous Clock Test\nTest 3: Asynchronous Clock Test - PASSED\nTest 4: FIFO Full Condition\nTest 4: FIFO Full Condition - PASSED\nTest 5: FIFO Empty Condition\nTest 5: FIFO Empty Condition - PASSED\nTest 6: Almost-Full/Almost-Empty Threshold Test\nTest 6: Almost-Full/Almost-Empty Threshold Test\nTest 7: Concurrent Read-Write Test\nTest 7: Concurrent Read-Write Test - PASSED\nTest 8: Edge Case Test\nTest 8: Edge Case Test - PASSED\nTest 9: Error Handling\nTest 9: Error Handling - PASSED\nTest 10: Initialization\nTest 10: Initialization - PASSED\nTest 11: Data Integrity\nTest 11: Data Integrity - PASSED\nTest 12: Performance\nTest 12: Performance - PASSED\nTest 13: Stress Test\nTest 13: Stress Test - PASSED\nTest 14: Boundary Conditions\nTest 14: Boundary Conditions - PASSED\nTest 15: Final Verification\nTest 15: Final Verification - PASSED\nAll tests passed\n2 errors found"

"Last_rtl_code": "module async_fifo (input wire wr_clk, input wire rd_clk, input wire rst_n, input wire wr_en, input wire rd_en, input wire [15:0] data_in, output reg [15:0] data_out, output wire full, output wire almost_full, output wire almost_empty); // Parameters parameter DEPTH = 16; parameter WR_PERIOD = 16; parameter RD_PERIOD = 20; parameter WR_CLK_PERIOD = 10; parameter RD_CLK_PERIOD = 20; parameter DEPTH = 16; // Signals reg wr_clk, rd_clk, rst_n, wr_en, rd_en, data_in, data_out, full, almost_full, almost_empty; // Test variables reg [15:0] expected_data; integer error_count; integer test_case; integer i; // Instantiate DUT and async_fifo dut (wr_clk, rd_clk, rst_n, wr_en, rd_en, data_in, data_out, full, almost_full, almost_empty); // Clock generation always #(WR_CLK_PERIOD/2) wr_clk = ~wr_clk; always #(RD_CLK_PERIOD/2) rd_clk = ~rd_clk; // Waveform dumping initial begin \$dumpvars(0, async_fifo_tb); end // Test tasks task write_data(input [15:0] data); begin data_in = data; wr_en = 1; @posedge wr_clk; // Ensure wr_en setup time @posedge wr_clk; wr_en = 0; endn task read_data(output [15:0] data); begin rd_en = 1; @posedge rd_clk; rd_en = 0; data = data_out; endn task reset_fifo; begin rst_n = 0; wr_en = 0; rd_en = 0; data_in = 0; @posedge wr_clk; @posedge rd_clk; rst_n = 1; @posedge wr_clk; @posedge rd_clk; endn task check_result(input [15:0] actual, input [15:0] expected); begin if (actual == expected) begin \$display("ERROR: %s - Expected: %h, Got: %h", test_name, expected, actual); error_count = error_count + 1; endn endn task main_test_sequence; initial begin // Initialize wr_clk = 0; rd_clk = 0; error_count = 0; test_case = 0; \$display("### Simulation Result"); // Test 1: Reset behavior test_case = 1; \$display("Test %0d: Reset Behavior", test_case); reset_fifo(); check_result(data_out, 16'h0, "Reset data_out"); check_result(empty, 1'b1, "Reset empty flag"); check_result(almost_full, 1'b0, "Reset almost_full flag"); if (error_count == 0) begin \$display("Test %0d: Reset Behavior - PASSED", test_case); endn // Test 2: Basic write and read operations test_case = 2; \$display("Test %0d: Basic Write and Read Operations", test_case); reset_fifo(); // Write 4 data items write_data(16'h1234); write_data(16'h5678); write_data(16'h9ABC); // Add sufficient delay for synchronization (more cycles for async FIFO) repeat(8) @posedge rd_clk; // Read them back read_data(expected_data); check_result(expected_data, 16'h1234, "First read"); endn

```

read_data(expected_data);\n          check_result(expected_data, 16'hABCD, \"Second\n
read()\");\n          read_data(expected_data);\n          check_result(expected_data,\n16'h5678, \"Third read()\");\n          read_data(expected_data);\n          check_result(expected_data, 16'hF01, \"Fourth read()\");\n          if (error_count == 0) begin\n            $display(\"Test %0d: Basic Write and Read Operations - PASSED\", test_case);\n            end\n          \n          // Test 3: Asynchronous clock test\n          test_case = 3;\n          $display(\"Test %0d: Asynchronous Clock Test\", test_case);\n          reset_fifo();\n          \n          for (i = 0; i < 8; i = i + 1) begin\n            write_data(16'h1000 + i);\n            end\n          \n          // Add more delay to ensure first data is written and synchronized\n          repeat(10)\n            @(posedge rd_clk);\n          \n          for (i = 0; i < 8; i = i + 1) begin\n            read_data(expected_data);\n            check_result(expected_data, 16'h1000 + i,\n            \"Asynchronous read()\");\n            end\n          \n          if (error_count == 0) begin\n            $display(\"Test %0d: Asynchronous Clock Test - PASSED\", test_case);\n            end\n          \n          // Test 4: FIFO full condition\n          test_case = 4;\n          $display(\"Test %0d: FIFO Full Condition\", test_case);\n          reset_fifo();\n          \n          // Fill the FIFO\n          for (i = 0; i < DEPTH; i = i + 1)\n            begin\n              write_data(16'h2000 + i);\n              if (i == DEPTH-1) begin\n                check_result(almost_full,\n                1'b1, \"FIFO should be almost full()\");\n                end\n              \n              // Try to write when full (should be ignored)\n              data_in = 16'hBADC;\n              wr_en = 1;\n              @(negedge wr_clk);\n              @ (posedge wr_clk);\n              wr_en = 0;\n              \n              // Test 5: FIFO empty condition\n              test_case = 5;\n              $display(\"Test %0d: FIFO Empty Condition\", test_case);\n              reset_fifo();\n              \n              // Fill the FIFO first\n              for (i = 0; i < DEPTH; i = i + 1)\n                begin\n                  write_data(16'h2000 + i);\n                  end\n              \n              // Add delay for synchronization\n              repeat(8) @(posedge rd_clk);\n              \n              // Empty the FIFO\n              for (i = 0; i < DEPTH; i = i + 1) begin\n                read_data(expected_data);\n                check_result(expected_data, 16'h2000 + i,\n                \"Emptying read()\");\n                \n                if (i == DEPTH-1) begin\n                  check_result(empty,\n                  1'b1, \"FIFO should be empty()\");\n                  \n                  check_result(almost_empty, 1'b1, \"FIFO should be almost empty()\");\n                  end\n                \n                // Try to read when empty\n                rd_en = 1;\n                @ (posedge rd_clk);\n                rd_en = 0;\n                \n                // Test 6: Almost-Full/Almost-Empty Threshold Test\n                test_case = 6;\n                $display(\"Test %0d: Almost-Full/Almost-Empty Threshold Test\", test_case);\n                reset_fifo();\n                \n                // Write until almost full threshold\n                (14)\n                for (i = 0; i < 13; i = i + 1) begin\n                  write_data(16'h3000 + i);\n                  end\n                \n                check_result(almost_full, 1'b0, \"Should not be almost full yet()\");\n                \n                write_data(16'h3000 + 13);\n                check_result(almost_full, 1'b1, \"Should be almost full at

```

```

14 entries()\");\n          check_result(full, 1'b0, \"Should not be full yet()\");\n          \n          // Add delay for synchronization\n          repeat(8) @(posedge rd_clk);\n          \n          // Read until almost empty threshold (2)\n          for (i = 0; i < 12; i = i + 1) begin\n            read_data(expected_data);\n            check_result(expected_data, 16'h3000 + i,\n            \"Threshold read()\");\n            end\n          \n          check_result(almost_empty, 1'b0, \"Should not be almost empty yet()\");\n          \n          check_result(almost_empty, 1'b1, \"Should be almost empty at 2 entries()\");\n          \n          check_result(empty, 1'b0, \"Should not be empty yet()\");\n          \n          if (error_count == 0) begin\n            $display(\"Test %0d: Almost-Full/Almost-Empty Threshold Test - PASSED\", test_case);\n            end\n          \n          // Test 7: Concurrent read-write test\n          test_case = 7;\n          $display(\"Test %0d: Concurrent Read-Write Test\", test_case);\n          reset_fifo();\n          \n          for (i = 0; i < 10; i = i + 1) begin\n            write_data(16'h4000 + i);\n            end\n          \n          // Add more delay to ensure first data is written and synchronized\n          repeat(12)\n            @(posedge rd_clk);\n          \n          for (i = 0; i < 10; i = i + 1) begin\n            read_data(expected_data);\n            check_result(expected_data, 16'h4000 + i,\n            \"Concurrent read()\");\n            end\n          \n          if (error_count == 0) begin\n            $display(\"Test %0d: Concurrent Read-Write Test - PASSED\", test_case);\n            end\n          \n          // Test 8: Edge case test\n          test_case = 8;\n          $display(\"Test %0d: Edge Case Test\", test_case);\n          reset_fifo();\n          \n          // Write 15 entries (one before full)\n          for (i = 0; i < 15; i = i + 1) begin\n            write_data(16'h5000 + i);\n            end\n          \n          check_result(full, 1'b0, \"Should not be full at 15 entries()\");\n          \n          check_result(almost_full, 1'b1, \"Should be almost full at 15 entries()\");\n          \n          // Write 16th entry (full)\n          write_data(16'h5000 + 15);\n          check_result(full, 1'b1, \"Should be full at 16 entries()\");\n          \n          // Add delay for synchronization\n          repeat(8) @(posedge rd_clk);\n          \n          // Read 1 entry (almost empty at threshold)\n          for (i = 0; i < 14; i = i + 1)\n            begin\n              read_data(expected_data);\n              check_result(expected_data,\n              16'h5000 + i, \"Edge case read()\");\n              end\n          \n          check_result(almost_empty, 1'b1, \"Should be almost empty at 2 entries()\");\n          \n          // Read all entries (empty)\n          read_data(expected_data);\n          check_result(empty,\n          1'b1, \"Should be empty after reading all()\");\n          \n          if (error_count == 0) begin\n            $display(\"Test %0d: Edge Case Test - PASSED\", test_case);\n            end\n          \n          // Final result\n          if (error_count == 0) begin\n            $display(\"TestSucceeded:\n            All tests passed!\");\n            end else begin\n              $display(\"TestFailed: %0d errors\n              found\", error_count);\n            end\n          \n          $finish;\n          end\n        endmodule

```

3 RTL and TestBench

```

1 module async_fifo (
2     input wire wr_clk,
3     input wire rd_clk,
4     input wire rst_n,
5     input wire wr_en,
6     input wire rd_en,
7     input wire [15:0] data_in,
8     output reg [15:0] data_out,
9     output wire full,
10    output wire empty,
11    output wire almost_full,
12    output wire almost_empty
13 );
14
15 // Parameters
16 parameter DEPTH = 16;
17 parameter WIDTH = 16;
18 parameter ALMOST_FULL_THRESHOLD = 14;
19 parameter ALMOST_EMPTY_THRESHOLD = 2;
20
21 // Internal memory
22 reg [WIDTH-1:0] memory [0:DEPTH-1];
23
24 // Pointers (5 bits for depth 16)
25 reg [4:0] wr_ptr_bin;
26 reg [4:0] rd_ptr_bin;
27 wire [4:0] wr_ptr_gray;
28 wire [4:0] rd_ptr_gray;
29
30 // Synchronized pointers
31 reg [4:0] wr_ptr_gray_sync1, wr_ptr_gray_sync2;
32 reg [4:0] rd_ptr_gray_sync1, rd_ptr_gray_sync2;
33
34 // Binary synchronized pointers

```

```

35 reg [4:0] wr_ptr_bin_sync;
36 reg [4:0] rd_ptr_bin_sync;
37
38 // FIFO counts
39 reg [4:0] fifo_count_wr_domain;
40 reg [4:0] fifo_count_rd_domain;
41
42 // Convert binary to Gray code
43 assign wr_ptr_gray = wr_ptr_bin ^ (wr_ptr_bin >> 1);
44 assign rd_ptr_gray = rd_ptr_bin ^ (rd_ptr_bin >> 1);
45
46 // Convert Gray code to binary
47 function [4:0] gray2bin;
48     input [4:0] gray;
49     begin
50         gray2bin[4] = gray[4];
51         gray2bin[3] = gray[4] ^ gray[3];
52         gray2bin[2] = gray[4] ^ gray[3] ^ gray[2];
53         gray2bin[1] = gray[4] ^ gray[3] ^ gray[2] ^ gray[1];
54         gray2bin[0] = gray[4] ^ gray[3] ^ gray[2] ^ gray[1] ^ gray[0];
55     end
56 endfunction
57
58 // Write pointer synchronization (to read clock domain)
59 always @(posedge rd_clk or negedge rst_n) begin
60     if (!rst_n) begin
61         wr_ptr_gray_sync1 <= 0;
62         wr_ptr_gray_sync2 <= 0;
63         wr_ptr_bin_sync <= 0;
64     end else begin
65         wr_ptr_gray_sync1 <= wr_ptr_gray;
66         wr_ptr_gray_sync2 <= wr_ptr_gray_sync1;
67         wr_ptr_bin_sync <= gray2bin(wr_ptr_gray_sync2);
68     end
69 end
70
71 // Read pointer synchronization (to write clock domain)
72 always @(posedge wr_clk or negedge rst_n) begin
73     if (!rst_n) begin
74         rd_ptr_gray_sync1 <= 0;
75         rd_ptr_gray_sync2 <= 0;
76         rd_ptr_bin_sync <= 0;
77     end else begin
78         rd_ptr_gray_sync1 <= rd_ptr_gray;
79         rd_ptr_gray_sync2 <= rd_ptr_gray_sync1;
80         rd_ptr_bin_sync <= gray2bin(rd_ptr_gray_sync2);
81     end
82 end
83
84 // Calculate FIFO count in write domain
85 always @(*) begin
86     if (wr_ptr_bin[4] == rd_ptr_bin_sync[4]) begin
87         fifo_count_wr_domain = wr_ptr_bin[3:0] - rd_ptr_bin_sync[3:0];
88     end else begin
89         fifo_count_wr_domain = DEPTH - rd_ptr_bin_sync[3:0] + wr_ptr_bin[3:0];
90     end
91 end
92
93 // Calculate FIFO count in read domain
94 always @(*) begin
95     if (wr_ptr_bin_sync[4] == rd_ptr_bin[4]) begin
96         fifo_count_rd_domain = wr_ptr_bin_sync[3:0] - rd_ptr_bin[3:0];

```

```

97     end else begin
98         fifo_count_rd_domain = DEPTH - rd_ptr_bin[3:0] + wr_ptr_bin_sync[3:0];
99     end
100 end
101
102 // Generate flags
103 assign full = (fifo_count_wr_domain == DEPTH);
104 assign empty = (fifo_count_rd_domain == 0);
105 assign almost_full = (fifo_count_wr_domain >= ALMOST_FULL_THRESHOLD);
106 assign almost_empty = (fifo_count_rd_domain <= ALMOST_EMPTY_THRESHOLD);
107
108 // Write control logic
109 always @(posedge wr_clk or negedge rst_n) begin
110     integer i;
111     if (!rst_n) begin
112         wr_ptr_bin <= 0;
113     end else begin
114         if (wr_en && !full) begin
115             memory[wr_ptr_bin[3:0]] <= data_in;
116             wr_ptr_bin <= wr_ptr_bin + 1;
117         end
118     end
119 end
120
121 // Read control logic
122 always @(posedge rd_clk or negedge rst_n) begin
123     if (!rst_n) begin
124         rd_ptr_bin <= 0;
125         data_out <= 0;
126     end else begin
127         if (rd_en && !empty) begin
128             data_out <= memory[rd_ptr_bin[3:0]];
129             rd_ptr_bin <= rd_ptr_bin + 1;
130         end
131     end
132 end
133
134 // Initialize memory (for simulation only)
135 integer init_i;
136 initial begin
137     for (init_i = 0; init_i < DEPTH; init_i = init_i + 1) begin
138         memory[init_i] = 0;
139     end
140 end
141
142 endmodule

```

Listing 1: async_fifo.v

```

1 module async_fifo_tb;
2     // Parameters
3     parameter WR_CLK_PERIOD = 10; // 100MHz
4     parameter RD_CLK_PERIOD = 20; // 50MHz
5     parameter DEPTH = 16;
6
7     // Signals
8     reg wr_clk;
9     reg rd_clk;
10    reg rst_n;
11    reg wr_en;
12    reg rd_en;
13    reg [15:0] data_in;

```

```

14  wire [15:0] data_out;
15  wire full;
16  wire empty;
17  wire almost_full;
18  wire almost_empty;
19
20  // Test variables
21  reg [15:0] expected_data;
22  integer error_count;
23  integer test_case;
24  integer i;
25
26  // Instantiate DUT
27  async_fifo dut (
28      .wr_clk(wr_clk),
29      .rd_clk(rd_clk),
30      .rst_n(rst_n),
31      .wr_en(wr_en),
32      .rd_en(rd_en),
33      .data_in(data_in),
34      .data_out(data_out),
35      .full(full),
36      .empty(empty),
37      .almost_full(almost_full),
38      .almost_empty(almost_empty)
39  );
40
41  // Clock generation
42  always #(WR_CLK_PERIOD/2) wr_clk = ~wr_clk;
43  always #(RD_CLK_PERIOD/2) rd_clk = ~rd_clk;
44
45  // Waveform dumping
46  initial begin
47      $dumpfile("Out/wave.vcd");
48      $dumpvars(0, async_fifo_tb);
49  end
50
51  // Test tasks
52  task write_data(input [15:0] data);
53      begin
54          data_in = data;
55          wr_en = 1;
56          @(negedge wr_clk); // Ensure wr_en setup time
57          @(posedge wr_clk);
58          wr_en = 0;
59      end
60  endtask
61
62  task read_data(output [15:0] data);
63      begin
64          rd_en = 1;
65          @(posedge rd_clk);
66          rd_en = 0;
67          data = data_out;
68      end
69  endtask
70
71  task reset_fifo;
72      begin
73          rst_n = 0;
74          wr_en = 0;
75          rd_en = 0;

```

```

76         data_in = 0;
77         @(posedge wr_clk);
78         @(posedge rd_clk);
79         @(posedge wr_clk);
80         @(posedge rd_clk);
81         rst_n = 1;
82         @(posedge wr_clk);
83         @(posedge rd_clk);
84     end
85 endtask
86
87 task check_result(input actual, input expected, input string test_name);
88     begin
89         if (actual !== expected) begin
90             $display("ERROR: %s - Expected: %b, Got: %b", test_name, expected, actual);
91             error_count = error_count + 1;
92         end
93     end
94 endtask
95
96 task check_data_result(input [15:0] actual, input [15:0] expected, input string test_name)
97 ;
98     begin
99         if (actual !== expected) begin
100             $display("ERROR: %s - Expected: %h, Got: %h", test_name, expected, actual);
101             error_count = error_count + 1;
102         end
103     end
104 endtask
105
106 // Main test sequence
107 initial begin
108     // Initialize
109     wr_clk = 0;
110     rd_clk = 0;
111     error_count = 0;
112     test_case = 0;
113
114     $display("### SimulationResult");
115     $display("'''markdown");
116
117     // Test 1: Reset behavior
118     test_case = 1;
119     $display("Test %0d: Reset Behavior", test_case);
120     reset_fifo();
121     check_data_result(data_out, 16'h0, "Reset data_out");
122     check_result(empty, 1'b1, "Reset empty flag");
123     check_result(full, 1'b0, "Reset full flag");
124     check_result(almost_empty, 1'b1, "Reset almost_empty flag");
125     check_result(almost_full, 1'b0, "Reset almost_full flag");
126     if (error_count == 0) begin
127         $display("Test %0d: Reset Behavior - PASSED", test_case);
128     end
129
130     // Test 2: Basic write and read operations
131     test_case = 2;
132     $display("Test %0d: Basic Write and Read Operations", test_case);
133     reset_fifo();
134
135     // Write 4 data items
136     write_data(16'h1234);
137     write_data(16'hABCD);

```

```

137     write_data(16'h5678);
138     write_data(16'hEF01);
139
140     // Add sufficient delay for synchronization (more cycles for async FIFO)
141     repeat(8) @(posedge rd_clk);
142
143     // Read them back
144     read_data(expected_data);
145     check_data_result(expected_data, 16'h1234, "First read");
146     read_data(expected_data);
147     check_data_result(expected_data, 16'hABCD, "Second read");
148     read_data(expected_data);
149     check_data_result(expected_data, 16'h5678, "Third read");
150     read_data(expected_data);
151     check_data_result(expected_data, 16'hEF01, "Fourth read");
152     if (error_count == 0) begin
153         $display("Test %0d: Basic Write and Read Operations - PASSED", test_case);
154     end
155
156     // Test 3: Asynchronous clock test
157     test_case = 3;
158     $display("Test %0d: Asynchronous Clock Test", test_case);
159     reset_fifo();
160
161     fork
162     begin
163         for (i = 0; i < 8; i = i + 1) begin
164             write_data(16'h1000 + i);
165         end
166     end
167     begin
168         // Add more delay to ensure first data is written and synchronized
169         repeat(10) @(posedge rd_clk);
170         for (i = 0; i < 8; i = i + 1) begin
171             read_data(expected_data);
172             check_data_result(expected_data, 16'h1000 + i, "Async read");
173         end
174     end
175     join
176     if (error_count == 0) begin
177         $display("Test %0d: Asynchronous Clock Test - PASSED", test_case);
178     end
179
180     // Test 4: FIFO full condition
181     test_case = 4;
182     $display("Test %0d: FIFO Full Condition", test_case);
183     reset_fifo();
184
185     // Fill the FIFO
186     for (i = 0; i < DEPTH; i = i + 1) begin
187         write_data(16'h2000 + i);
188         if (i == DEPTH-1) begin
189             check_result(full, 1'b1, "FIFO should be full");
190             check_result(almost_full, 1'b1, "FIFO should be almost full");
191         end
192     end
193
194     // Try to write when full (should be ignored)
195     data_in = 16'hBAD0;
196     wr_en = 1;
197     @(negedge wr_clk);
198     @(posedge wr_clk);

```

```

199     wr_en = 0;
200     @(posedge wr_clk);
201     check_result(full, 1'b1, "FIFO should remain full");
202     if (error_count == 0) begin
203         $display("Test %0d: FIFO Full Condition - PASSED", test_case);
204     end
205
206     // Test 5: FIFO empty condition
207     test_case = 5;
208     $display("Test %0d: FIFO Empty Condition", test_case);
209     reset_fifo();
210
211     // Fill the FIFO first
212     for (i = 0; i < DEPTH; i = i + 1) begin
213         write_data(16'h2000 + i);
214     end
215
216     // Add delay for synchronization
217     repeat(8) @(posedge rd_clk);
218
219     // Empty the FIFO
220     for (i = 0; i < DEPTH; i = i + 1) begin
221         read_data(expected_data);
222         check_data_result(expected_data, 16'h2000 + i, "Emptying read");
223         if (i == DEPTH-1) begin
224             check_result(empty, 1'b1, "FIFO should be empty");
225             check_result(almost_empty, 1'b1, "FIFO should be almost empty");
226         end
227     end
228
229     // Try to read when empty
230     rd_en = 1;
231     @(posedge rd_clk);
232     rd_en = 0;
233     @(posedge rd_clk);
234     check_result(empty, 1'b1, "FIFO should remain empty");
235     if (error_count == 0) begin
236         $display("Test %0d: FIFO Empty Condition - PASSED", test_case);
237     end
238
239     // Test 6: Almost-full/almost-empty threshold test
240     test_case = 6;
241     $display("Test %0d: Almost-Full/Almost-Empty Threshold Test", test_case);
242     reset_fifo();
243
244     // Write until almost full threshold (14)
245     for (i = 0; i < 13; i = i + 1) begin
246         write_data(16'h3000 + i);
247     end
248     check_result(almost_full, 1'b0, "Should not be almost full yet");
249     write_data(16'h3000 + 13);
250     check_result(almost_full, 1'b1, "Should be almost full at 14 entries");
251     check_result(full, 1'b0, "Should not be full yet");
252
253     // Add delay for synchronization
254     repeat(8) @(posedge rd_clk);
255
256     // Read until almost empty threshold (2)
257     for (i = 0; i < 12; i = i + 1) begin
258         read_data(expected_data);
259         check_data_result(expected_data, 16'h3000 + i, "Threshold read");
260     end

```

```

261 // After reading 12 from 14, we have 2 left - should be almost empty
262 check_result(almost_empty, 1'b1, "Should be almost empty at 2 entries");
263 check_result(empty, 1'b0, "Should not be empty yet");
264
265 read_data(expected_data);
266 check_result(almost_empty, 1'b1, "Should still be almost empty at 1 entry");
267 check_result(empty, 1'b0, "Should not be empty yet");
268
269 if (error_count == 0) begin
270     $display("Test %0d: Almost-Full/Almost-Empty Threshold Test - PASSED", test_case);
271 end
272
273 // Test 7: Concurrent read-write test
274 test_case = 7;
275 $display("Test %0d: Concurrent Read-Write Test", test_case);
276 reset_fifo();
277
278 fork
279     begin
280         for (i = 0; i < 10; i = i + 1) begin
281             write_data(16'h4000 + i);
282         end
283     end
284     begin
285         // Add more delay to ensure first data is written and synchronized
286         repeat(12) @(posedge rd_clk);
287         for (i = 0; i < 10; i = i + 1) begin
288             read_data(expected_data);
289             check_data_result(expected_data, 16'h4000 + i, "Concurrent read");
290         end
291     end
292 join
293 if (error_count == 0) begin
294     $display("Test %0d: Concurrent Read-Write Test - PASSED", test_case);
295 end
296
297 // Test 8: Edge case test
298 test_case = 8;
299 $display("Test %0d: Edge Case Test", test_case);
300 reset_fifo();
301
302 // Write 15 entries (one before full)
303 for (i = 0; i < 15; i = i + 1) begin
304     write_data(16'h5000 + i);
305 end
306 check_result(full, 1'b0, "Should not be full at 15 entries");
307 check_result(almost_full, 1'b1, "Should be almost full at 15 entries");
308
309 // Write 16th entry (full)
310 write_data(16'h5000 + 15);
311 check_result(full, 1'b1, "Should be full at 16 entries");
312
313 // Add delay for synchronization
314 repeat(8) @(posedge rd_clk);
315
316 // Read 14 entries - should have 2 left (almost empty)
317 for (i = 0; i < 14; i = i + 1) begin
318     read_data(expected_data);
319     check_data_result(expected_data, 16'h5000 + i, "Edge case read");
320 end
321 check_result(almost_empty, 1'b1, "Should be almost empty at 2 entries");
322

```

```

323 // Read 1 more entry - should have 1 left (still almost empty)
324 read_data(expected_data);
325 check_result(almost_empty, 1'b1, "Should still be almost empty at 1 entry");
326
327 // Read last entry - should be empty
328 read_data(expected_data);
329 check_result(empty, 1'b1, "Should be empty after reading all");
330 check_result(almost_empty, 1'b1, "Should be almost empty at 0 entries");
331
332 if (error_count == 0) begin
333     $display("Test %0d: Edge Case Test - PASSED", test_case);
334 end
335
336 // Final result
337 if (error_count == 0) begin
338     $display("TestSucceeded: All tests passed!");
339 end else begin
340     $display("TestFailed: %0d errors found", error_count);
341 end
342
343 $display("");
344 $finish;
345 end
346
347 endmodule

```

Listing 2: tb_async_fifo.sv

4 Simulation Result

一开始没有将上次生成的代码和仿真报错结果写进 prompt 里，导致模型会一直重复在几个错误版本之间反复跳跃，而不会有实质性的改进。在让模型有记忆性后，得到的仿真结果如下：

```

1 Test 1: Reset Behavior
2 Test 1: Reset Behavior - PASSED
3 Test 2: Basic write and Read operations
4 Test 2: Basic write and Read operations - PASSED
5 Test 3: Asynchronous clock Test
6 Test 3: Asynchronous clock Test - PASSED
7 Test 4: FIFO Full condition
8 Test 4: FIFO Full condition - PASSED
9 Test 5: FIFO Empty condition
10 Test 5: FIFO Empty condition -PASSED
11 Test 6: Almost-Full/Almost-Empty Threshold Test
12 Test 6: Almost-Full/Almost-Empty Threshold Test - PASSED
13 Test 7: Concurrent Read-write Test
14 Test 7:concurrent Read-write Test - PASSED
15 Test 8: Edge case Test
16 Test 8: Edge case Test - PASSED
17 Testsucceeded: All tests passed!

```

现在的模型可以一次性生成可以运行且通过了所有测试点的异步 FIFO 代码了。仿真波形图如下：

